

PROGRAMMER // 2026

# プログラム説明資料

KuboEngine / Octopus

DirectX 12 × C++ × HLSL。設計、データ駆動、0BB 衝突判定、シャドウマップ、負荷対策を中心に構成。

C++

DirectX 12

HLSL

ImGui

CSV Driven

Performance

AUTHOR

TAKU OKUBO / オオクボ タク

ゲームプログラマー志望 / 日本工学院専門学校 ゲームクリエイター科 4年制

# 目次

技術的に工夫した点、制作情報、担当範囲、使用ライブラリを整理。

## 01 制作情報

期間 / 人数 / 担当箇所 / 使用ライブラリ

## 03 作品概要

KuboEngine / Octopus のゲーム構成

## 05 ゲーム進行

GameplayFlowController による状態管理

## 07 敵・衝突・性能

空間分割、OBB、Telemetry

## 09 データ駆動

CSV による調整基盤

## 11 描画・リソース

Shadow Map、演出、キャッシュ

## 02 技術的な注力点

設計、データ駆動、負荷対策

## 04 アーキテクチャ

SceneFactory と SessionContext

## 06 プレイヤー・武器

成長と 4 種類の武器制御

## 08 レベルアップ

候補生成と CSV 重み調整

## 10 UI / Debug / 演出

HUD、MiniMap、ImGui、Presentation

## 12 まとめ

学びと今後の改善

# 制作情報 / 担当範囲

制作形態、担当箇所、使用した外部ライブラリを明記。

## 制作期間

2025/07～開発中（継続制作）  
延べ時間：約350～400時間

## 制作人数

個人制作  
プログラマ 1 名のみ

## 担当箇所

コード全般を担当。  
engine/、game/directxgame/、Resources/DirectXGame/data/ などの実装・調整を担当。

## 使用ライブラリ / 外部コード

DirectX 12 / HLSL  
Dear ImGui / ImGuizmo / imgui-node-editor / ImPlot  
DirectXTex / Assimp / FontAwesome IconFont  
Windows API / XAudio 系の音再生基盤  
学内配布・外部導入コードを含む部分は externals/ 配下に分離

## 制作で意識したこと

- ゲーム性だけでなく、仕様変更能耐えるクラス分割を意識して実装。
- 数値調整を CSV に分離し、コード変更なしでバランス調整できる状態を構築。
- デバッグ UI とテレメトリで、感覚だけではなく数値を見て改善。

# 技術的な注力ポイント

設計、描画、衝突判定、調整環境まで、現行プログラムで工夫した点を整理。

## 描画・表現

- Shadow Map でプレイヤーと敵の影を描画し、画面の接地感を強化。
- 影の濃さ、ぼかし、bias、範囲を SceneLighting から調整可能にした。
- ポストエフェクトと演出クラスを分け、見た目の変更範囲を限定。

## 衝突・負荷対策

- 空間分割で近傍候補を絞り、敵数が増えても総当たりを避ける。
- Player、Enemy、Bullet を OBB で判定し、回転やスケールを反映。
- Telemetry で敵数、弾数、EXP Orb 数の最大数と削減数を監視。

## 調整・検証基盤

- CSV で武器強化、敵出現、レベルアップ重み、UI 配置を外部化。
- ImGui と DebugPanel で実行中に状態確認と調整を行える。
- Line::DrawOBB と ColliderManager で判定形状を可視化。

# KuboEngine / Octopus

敵を倒して EXP を集め、レベルアップでビルドを強化していく 3D サバイバルシューティング。

## Genre

3D Shooting / Survival Action

## Role

1人制作 / コード全般

## Engine

自作 DirectX 12 エンジン

## Data

CSV によるバランス・UI 調整

## 現在の主なゲーム要素

### Scene / Flow

Title / Game / Result とゲーム中状態を分けて管理。

### Combat

通常弾、衛星弾、ドローン、雷撃を統合。

### Growth

状態に応じてレベルアップ候補を生成し、PlayerManager へ適用。

### Enemy

敵生成、衝突、EXP Orb、ボス状態を管理。

### Presentation

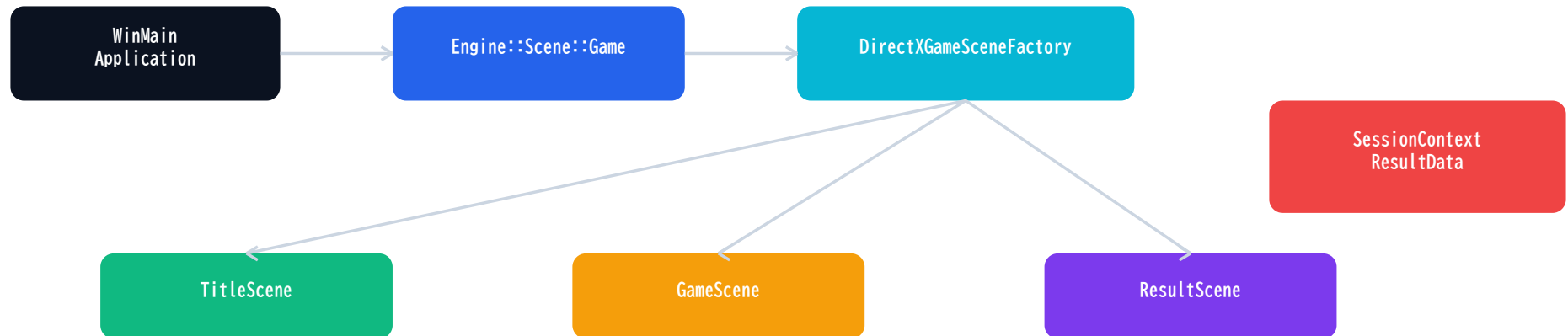
Boss、戦闘、死亡、遷移演出を専用クラスへ分離。

## ゲームループの概要



# シーン構成と責務分離

OS エントリーポイントから Game、SceneFactory、各 Scene へ責務を渡す構成。



## 設計上の工夫

- main.cpp は Framework 実行に集中し、具体的なシーン生成は Factory へ委譲。
- SessionContext が run count、scene visit count、result summary を保持。
- GameScene は集約 root とし、進行・UI・演出・敵・武器の責務は専用クラスへ分離。

# GameplayFlowController

ゲーム中の状態を `enum class GameState` で管理し、入力・UI・世界更新の可否を状態から判断。



## 状態管理の工夫

- プレイ中・ポーズ中・レベルアップ中で、ワールド更新、HUD 表示、カーソル表示を切替。
- ボス登場と撃破演出をゲーム本体から分離し、カメラやエフェクト制御を専用化。
- 死亡時は `PlayerDeathPresentation` と `ResultScene` への遷移を連携。

# 成長と武器制御

PlayerManager は HP / EXP / Lv / 武器状態の窓口。内部で Progression と WeaponController に分割。



## 武器実装の工夫

- 武器ごとの処理は PlayerWeaponController に集約し、PlayerManager は外部公開 API と成長処理の窓口にした。
- 通常弾・衛星弾・ドローン・雷撃を同じ成長ルールで扱い、拡張しやすくした。
- weaponUpgradeSettings.csv により Lv ごとの弾数、半径、間隔、貫通、ダメージ補正を調整可能にした。



# 敵管理と OBB 衝突判定

EnemyManager は敵リストと EXP Orb を持ち、生成・衝突・撃破ドロップ・ボス状態を統合する。

## EnemySpawnController

CSV から敵タイプ・スポーン設定を読み込み、経過時間に応じて出現数や間隔を制御。

## Spatial Hash

spatialMap と nearbyEnemies で候補を近傍セルに限定し、総当たりを避ける。

## OBB Narrow Phase

Player、Enemy、通常弾、衛星弾などをモデル由来の OBB で判定。回転やスケールを反映しやすい。

## Collider Debug

Line::DrawOBB と ColliderManager で判定形状を表示し、見た目と当たり判定のズレを確認。

## Telemetry

敵数、EXP Orb 数、弾数 peak / prune を取得し、デバッグ UI と CSV 出力で負荷を可視化。

## 処理フロー

Spawn

SpatialMap

OBB Check

Damage

Drop / Prune

# LevelUpChoiceService

候補生成と適用処理をサービス化。武器所持・レベル・HP 状態に応じて候補を切り替える。

## 現行構造

```
enum class LevelUpUpgrade {
    Normal, Orbit, Drone, Lightning,
    Attack, MaxHp, MoveSpeed, Heal
};

struct LevelUpChoice {
    LevelUpUpgrade upgrade;
    std::string texturePath;
    std::string iconPath;
};
```

候補生成と適用処理を分け、武器追加や回復候補の調整をコード全体へ波及させにくくした。

## 候補決定の流れ

### Player 状態

HP、所持武器、各武器 Lv、上限到達を参照

### Pool 生成

未所持武器は解禁、所持済みは強化候補へ切替

### Weight

levelupWeights.csv で出現比率を調整

### Apply

PlayerManager の Upgrade / Add / Heal API を呼び出し

# データ駆動型の調整基盤

調整値を CSV に分離し、コンパイルなしでバランス・UI・リソースを変更できるようにしている。

## playerStatus.csv

初期 HP、攻撃力、移動速度、上限値

## weaponUpgradeSettings.csv

武器 Lv ごとの弾数、間隔、半径、貫通

## levelupWeights.csv

選択肢の出現重み、HP 状況による回復重み

## enemyTypes.csv

敵タイプごとの HP、速度、EXP、出現数

## enemySpawnSettings.csv

最大敵数 84、スポーン距離、加速、間隔

## ui\_layout\_\*.csv

タイトル、HUD、ポーズ、結果画面の配置

## resource\_manifest.csv

テクスチャ、音、データファイルのロード対象

## debug\_tuning.csv

デバッグ調整値と検証用パラメータ

## 調整しやすくする工夫

- マジックナンバーを減らし、数値の意味をファイル単位で追える。
- 調整担当がコードを触らずにバランスや UI 配置を変更できる。
- resource\_manifest.csv により、ロード対象を一覧化し抜け漏れを検出しやすい。

## 調整フロー



# HUD・ミニマップ・調整環境

スプライトベースの UI 基盤と ImGui デバッグ UI を使い、実行中の状態確認と調整をやすくした。

## UIElement

位置、サイズ、表示、親子関係、縦横レイアウトを共通化。

## GameplayHudPresentation

Timer、HP、EXP、KeyUI、MiniMap、開始/死亡/被弾オーバーレイを管理。

## MiniMap

敵・EXP Orb・プレイヤーを円形範囲にクランプして表示。

## GameplayDebugUiController

各 DebugPanel の統合とデバッグ操作の窓口。

## Collider / OBB Debug

Player、Enemy、Bullet の OBB を線描画し、判定範囲を実行中に確認。

## Runtime / Statistics

オブジェクト数、敵数、弾数、EXP Orb 数を監視。

## 開発効率化の工夫

- DockSpace と Scene Window により、実行画面を見ながら数値・描画・音を同時に調整できる。
- DebugPanel を機能ごとに分け、検証したい対象へ素早くアクセスできるようにした。

## 確認サイクル



# シャドウマップ・演出・リソース管理

描画基盤、影描画、ゲーム演出、ロード処理を分け、見た目と開発効率の両方を扱いやすくした。

## Shadow Map Pass

2048 x 2048 の深度マップを生成。BeginShadowPass から Draw Shadow を呼び、通常描画で参照。

## Shadow Tuning

SceneLighting で有効化、濃さ、ぼかし、bias、描画範囲を調整。CSV調整とImGui操作に対応。

## OffscreenRenderManager

一度オフスクリーンへ描画し、ポストエフェクトやシーン表示へ利用。

## Post Effects

Fullscreen、GrayScale、Vignette、RadialBlur などを実装。

## Boss / Combat Presentation

登場、撃破、雷撃などの演出を専用クラスで制御。

## GameTexture / Model / Audio Cache

重複ロードを抑え、利用側コードを単純化。

## リソース管理の工夫

- ロードパスを ResourcePaths / DataPaths に集約し、Resources 配下を一貫して扱う。
- SceneTransition と CurtainTransition により、画面切り替えと状態遷移を自然につなぐ。

# まとめ / 今後の改善

DX12 の低レイヤ実装とゲーム固有ロジックをつなぎ、調整しやすい制作環境を作った。

## 学び 1: エンジン設計

Framework、Scene、描画、入力、音、リソースを分離し、ゲーム側が使う API を整理。

## 学び 2: データ駆動

CSV と ImGui によって、調整と実装を切り離す重要性を実感。

## 学び 3: 最適化

空間分割、上限管理、Telemetry で、大量オブジェクトの負荷を数値で判断。

## 学び 4: 完成優先

演出・UI・結果画面まで含め、遊べる流れとして成立させる判断力を強化。

## Future Goal

- タスクシステムや非同期ロードを導入し、DX12 の高度な機能活用へ広げる。
- 調整値保存・再読込をさらに統一し、制作サイクルを短縮する。
- プレゼンテーション層を継続的に整理し、演出追加時の影響範囲を狭める。

THANK YOU